



UX/UI 인터랙션 패턴 종합 가이드

웹 및 모바일 애플리케이션에서 우수한 사용자 경험을 제공하는
인터랙션 패턴을 체계적으로 정리한 레퍼런스 문서



11개 카테고리



50+ 패턴



UX 원칙 기반

목차

Table of Contents

- 01 마이크로 인터랙션** Micro-interactions
버튼 피드백, 토글, 로딩 상태, 당겨서 새로그침, 좋아요, 카운터

- 02 내비게이션 패턴** Navigation Patterns
탭 바, 햄버거 메뉴, 제스처 내비게이션, 브레드크럼, 바텀 시트, 스와이프

- 03 피드백 및 응답** Feedback & Response
햅틱, 스켈레톤 스크린, 낙관적 업데이트, 프로그레스, 토스트/스낵바

- 04 제스처 기반 인터랙션** Gesture-based Interactions
스와이프 액션, 핀치 투 줌, 드래그 앤 드롭, 롱 프레스, 더블 탭

- 05 스크롤 인터랙션** Scroll Interactions
패럴랙스, 무한 스크롤, 고정 헤더, 스크롤 트리거, 스크롤 스냅

- 06 폼 인터랙션** Form Interactions
인라인 검증, 플로팅 라벨, 자동완성, 스마트 기본값, 마법사, 입력 마스크

- 07 전환 및 애니메이션** Transition & Animation
페이지 전환, 공유 요소, 모핑, 스프링 물리학, 시차 애니메이션

- 08 온보딩** Onboarding
점진적 공개, 톨팁, 코치 마크, 빈 상태, 기능 발견

- 09 접근성 인터랙션** Accessibility Interactions
스크린 리더, 포커스 관리, 모션 감소, 색상 대비, 터치 타겟

- 10 딜라이트 패턴** Delight Patterns
이스터 에그, 축하 애니메이션, 게이미피케이션, 개인화, 사운드

11 다크 패턴 — 피해야 할 것들 Dark Patterns to Avoid

확인 수치심, 강제 연속, 트릭 질문, 숨겨진 비용, 방해 광고, 시각적 간섭, 강제 행동

SECTION 01

마이크로 인터랙션 Micro-interactions

마이크로 인터랙션은 단일 작업을 중심으로 하는 작고 정밀한 순간적 인터랙션이다. 트리거(Trigger) → 규칙(Rules) → 피드백(Feedback) → 루프/모드(Loops & Modes)의 4단계 구조로 구성된다 (Dan Saffer, 2013).

1.1 버튼 피드백 Button Feedback

설명	사용자가 버튼을 누를 때 시각적·촉각적 반응을 제공하는 패턴. 눌림 효과(scale down), 색상 변화, ripple effect 등이 포함된다.
UX 원칙	피드백 원칙 (Feedback Principle) — 시스템은 사용자의 모든 행동에 즉각적인 반응을 보여야 한다. 100ms 이내의 반응은 "즉각적"으로 인식된다 (Jakob Nielsen).
플랫폼	Both (Web + Mobile)
구현 복잡도	Low
실제 사례	Material Design의 Ripple Effect, iOS의 버튼 highlight state, Stripe의 버튼 로딩 애니메이션

구현 팁

- `transform: scale(0.95)`와 `transition: 150ms ease`를 결합하여 자연스러운 눌림 효과 구현
- `:active` pseudo-class에 시각 변화 적용, `:focus-visible`에는 포커스 링 유지
- 모바일에서는 `touch-action: manipulation`으로 300ms 탭 딜레이 제거
- 비활성 버튼에는 `opacity: 0.4 + pointer-events: none` 적용

1.2 토글 애니메이션 Toggle Animation

설명	ON/OFF 상태 전환 시 슬라이드, 색상 변화, 아이콘 모핑 등의 애니메이션으로 상태 변화를 명확히 전달하는 패턴.
UX 원칙	가시성 원칙 (Visibility of System Status) — Nielsen의 10가지 휴리스틱 중 1번. 현재 상태를 사용자에게 항상 알려야 한다.
플랫폼	Both
구현 복잡도	Low
실제 사례	iOS Settings 토글, Android Material Switch, Slack의 알림 토글

구현 팁

- 토글 전환 시간은 200~300ms가 적정
- 색상만으로 상태를 구분하지 말 것 — 아이콘이나 텍스트 라벨 병행 (색각 이상 대응)
- `prefers-reduced-motion` 미디어 쿼리로 모션 감소 설정 대응
- 토글 변경 시 서버 동기화가 필요하면 낙관적 업데이트(Optimistic Update) 적용

1.3 로딩 상태 Loading States

설명	데이터 로딩 중임을 사용자에게 알리는 시각적 표현. 스피너, 프로그레스 바, 스켈레톤 UI, 시머 효과 등 다양한 형태.
UX 원칙	도허티 임계값 (Doherty Threshold) — 시스템 응답이 400ms 이내일 때 생산성이 급격히 향상. 그 이상 걸릴 경우 진행 피드백이 필수.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Facebook/LinkedIn의 Skeleton Screen, YouTube의 시머 로딩, Notion의 페이지 로딩

구현 팁

- 0~100ms: 피드백 불필요 / 100~400ms: 간단한 인디케이터 / 400ms~1s: 스피너 / 1s 이상: 프로그레스 바 or 스켈레톤
- 스켈레톤 UI는 실제 콘텐츠 레이아웃과 동일한 형태로 만들어 인지 부하 감소
- 시머 효과는 왼쪽→오른쪽 gradient 애니메이션으로 "살아있음"을 전달
- 무한 스피너 대신 예상 시간 또는 단계별 메시지 표시

1.4 당겨서 새로고침 Pull-to-Refresh

설명	리스트 최상단에서 아래로 당기면 콘텐츠가 새로고침되는 모바일 네이티브 패턴. Loren Brichter가 Tweetie 앱에서 최초 발명(2008).
UX 원칙	직접 조작 원칙 (Direct Manipulation) — 인터페이스 객체를 직접 조작하는 느낌을 제공하면 학습 비용이 낮아진다.
플랫폼	Mobile
구현 복잡도	Medium
실제 사례	Twitter/X, Instagram 피드, Gmail 받은편지함, 네이버 앱

구현 팁

- 당김 거리에 비례하는 시각 피드백 제공 (아이콘 회전, 진행률 표시)
- 임계값 도달 전 놓으면 원위치 복귀, 도달 후 놓으면 새로고침 실행
- 새로고침 완료 후 최소 300ms 지연으로 "완료되었음"을 인지할 시간 확보
- 웹에서는 `overscroll-behavior: contain`으로 브라우저 기본 동작 방지

1.5 좋아요/하트 애니메이션 Like/Heart Animation

설명	좋아요·하트 버튼을 누를 때 파티클 폭발, 크기 변화, 색상 채움 등의 풍부한 애니메이션으로 감정적 만족감을 제공.
UX 원칙	감성 디자인 (Emotional Design) — Don Norman의 3단계 디자인 중 "본능적" 수준. 가변 보상 (Variable Reward) — 매번 미묘하게 다른 애니메이션이 도파민 반응을 촉진.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Twitter/X의 하트 폭발, Instagram 더블탭 하트, Medium의 Clap 카운터

구현 팁

- Lottie 또는 CSS keyframe으로 구현 (SVG path morphing + scale + opacity)
- 애니메이션 지속 시간은 300~600ms가 이상적
- 모바일에서 더블탭 좋아요는 피드 이미지 중앙에 큰 하트 오버레이로 구현
- 취소(unlike) 시에는 화려한 애니메이션 없이 조용히 처리 (비대칭 피드백)

1.6 카운터 애니메이션 Counter Animation

설명	숫자가 변경될 때 롤링(odometer), 슬라이드, 페이드 등의 애니메이션으로 변화를 강조하는 패턴.
UX 원칙	변화 맹시 방지 (Change Blindness Prevention) — 인간은 점진적 변화를 놓치기 쉬우므로 변화 순간에 시각적 강조가 필요.
플랫폼	Both
구현 복잡도	Low
실제 사례	Robinhood 주가 변동, 토스 자산 현황, GitHub 스타 카운터, 쿠팡 장바구니 뱃지

구현 팁

- 숫자 증가/감소 방향에 따라 위→아래 또는 아래→위 슬라이드 적용
- requestAnimationFrame으로 부드러운 숫자 카운팅 구현
- 큰 숫자 변화는 easing(ease-out)으로 점진적 감속 효과 적용
- 가격, 포인트 등 금전 관련 숫자는 천 단위 구분자와 함께 애니메이션

SECTION 02

내비게이션 패턴 Navigation Patterns

내비게이션은 사용자가 앱 내에서 목적지를 찾고 이동하는 방식을 결정한다. 좋은 내비게이션은 "투명"해야 한다 — 사용자가 내비게이션 자체를 의식하지 않고 콘텐츠에 집중할 수 있어야 한다.

2.1 탭 바 Tab Bar / 하단 탭 내비게이션

설명	화면 하단에 3~5개의 주요 섹션을 아이콘+라벨로 배치하는 최상위 내비게이션 패턴.
UX 원칙	피츠의 법칙 — 하단 탭은 엄지 도달 범위(thumb zone) 내에 위치. 밀러의 법칙 — 탭은 5개 이하로 제한.
플랫폼	Mobile
구현 복잡도	Low
실제 사례	Instagram, 카카오톡, YouTube, 토스

구현 팁

- 최소 터치 타겟 48x48dp (Material) 또는 44x44pt (iOS HIG) 준수
- 활성 탭은 색상 + 아이콘 변화(filled/outlined)로 명확히 구분
- 알림 배지는 최소 크기로 표시하되 접근성을 위해 `aria-label`에 카운트 포함
- 탭 전환 시 상태 보존 — 각 탭의 스크롤 위치, 입력 상태 유지

2.2 햄버거 메뉴 Hamburger Menu

설명	세 줄 아이콘(≡) 뒤에 내비게이션 메뉴를 숨기는 패턴. 화면 공간을 절약하지만 발견 가능성이 낮다.
UX 원칙	인지 부하 이론 — 정보를 숨겨 화면을 단순화하지만, "보이지 않으면 존재하지 않는 것" 원칙과 충돌.
플랫폼	Both (모바일 우선)
구현 복잡도	Low
실제 사례	Gmail (모바일), Spotify (과거), Facebook (과거)

구현 팁

- 주요 기능은 탭 바에 노출하고 부차적 기능만 햄버거 메뉴에 배치
- 메뉴 열림 시 오버레이(scrim) + 스와이프로 닫기 지원
- 메뉴 항목에 아이콘 병행으로 스캔 속도 향상
- 웹에서는 768px 이상에서 사이드바로 확장, 그 이하에서 햄버거로 축소

2.3 제스처 내비게이션 Gesture Navigation

설명	스와이프, 옛지 제스처 등 터치 동작으로 화면 간 이동하는 패턴.
UX 원칙	직접 조작 + 야콥의 법칙 — 사용자는 다른 앱에서 배운 제스처를 동일하게 기대한다.
플랫폼	Mobile
구현 복잡도	Medium
실제 사례	iOS edge swipe, Android 10+ 제스처, Tinder 스와이프

구현 팁

- 플랫폼 기본 제스처와 충돌하지 않도록 주의
- 제스처 시작 시 시각적 힌트 제공
- 제스처가 유일한 조작 방법이 되지 않도록 버튼 대안 항상 제공

2.4 브레드크럼 Breadcrumbs

설명	현재 페이지의 계층적 위치를 경로 형태로 보여주는 보조 내비게이션.
UX 원칙	인지 지도 (Cognitive Map) — 사용자가 정보 공간 내 자신의 위치를 파악. 헨젤과 그레텔 효과 .
플랫폼	Web
구현 복잡도	Low
실제 사례	Amazon, Google 검색 결과, Notion, Jira

구현 팁

- 마지막 항목(현재 페이지)은 링크가 아닌 일반 텍스트로 표시
- `<nav aria-label="Breadcrumb"> + <ol>` 시맨틱 마크업 사용
- 구조화 데이터(Schema.org BreadcrumbList) 추가로 SEO 향상
- 모바일에서는 "< 상위 카테고리" 형태의 단일 뒤로가기 링크로 대체

2.5 바텀 시트 Bottom Sheet

설명	화면 하단에서 올라오는 모달 패널. 추가 옵션, 상세 정보, 필터 등을 맥락을 유지하면서 제공.
UX 원칙	맥락 보존 — 전체 페이지 전환 없이 추가 정보를 제공. 피츠의 법칙 — 엄지 도달 범위 내에서 조작 가능.
플랫폼	Mobile
구현 복잡도	Medium
실제 사례	Google Maps, Uber, 카카오맵, 토스 결제 시트

구현 팁

- 3단계 높이 지원: peek(미리보기), half(절반), full(전체화면)
- 드래그 핸들(grab bar)을 상단에 배치
- 배경 스크림 + 바깥 영역 탭으로 닫기 지원
- 스크롤 가능한 콘텐츠와 시트 드래그 간 제스처 충돌 해결 필요

2.6 스와이프 내비게이션 Swipe Navigation

설명	좌우 스와이프로 탭이나 페이지 간 이동하는 패턴. 상단 탭 인디케이터와 결합.
UX 원칙	공간적 기억 — 콘텐츠를 공간적으로 배치하면 위치를 기억하기 쉽다. 직접 조작 .
플랫폼	Mobile
구현 복잡도	Medium
실제 사례	카카오톡, Tinder, iOS 날씨 앱, Chrome 모바일

구현 팁

- 스와이프 방향의 다음 페이지를 미리보기로 살짝 보여주어 "더 있음"을 암시
- 첫/마지막 페이지에서는 바운스 효과로 끝임을 전달
- 탭 인디케이터의 밑줄이 스와이프 제스처에 따라 함께 이동하는 연동 애니메이션 구현

SECTION 03

피드백 및 응답 Feedback & Response

시스템이 사용자의 행동에 적절히 반응하는 것은 신뢰 구축의 핵심이다. Nielsen의 휴리스틱 1번 "시스템 상태의 가시성 (Visibility of System Status)"은 모든 피드백 패턴의 근간이 된다.

3.1 햅틱 피드백 Haptic Feedback

설명	터치 인터랙션에 진동 반응을 결합하여 물리적 조작감을 제공하는 패턴.
UX 원칙	다중 감각 피드백 — 시각+촉각 결합 시 인지 정확도 향상. 어포던스 — 물리 버튼처럼 "눌린 느낌" 재현.
플랫폼	Mobile
구현 복잡도	Low
실제 사례	iPhone 키보드, Apple Watch Digital Crown, 토스 송금 완료 진동

구현 팁

- iOS: `UIImpactFeedbackGenerator` (light/medium/heavy)
- Android: `HapticFeedbackConstants.LONG_PRESS`, `VIRTUAL_KEY`
- 웹: `navigator.vibrate()` API (Android Chrome 지원, iOS Safari 미지원)
- 과도한 햅틱은 배터리 소모 + 사용자 피로 유발 — 중요 액션에만 선별 적용

3.2 스켈레톤 스크린 Skeleton Screen

설명	데이터 로딩 중 실제 UI 레이아웃의 빈 뼈대를 회색 블록으로 먼저 보여주는 패턴. 스피너보다 체감 로딩 시간을 단축.
UX 원칙	인지된 성능 (Perceived Performance) — 실제 속도가 아닌 사용자가 "느끼는" 속도가 중요. 스켈레톤은 콘텐츠가 곧 나타날 것이라는 기대감을 형성.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Facebook, LinkedIn, YouTube, 네이버 뉴스, 배달의민족

구현 팁

- 실제 콘텐츠와 동일한 레이아웃으로 스켈레톤 구성
- 시머 애니메이션: `linear-gradient(90deg, #f0f0f0, #e0e0e0, #f0f0f0)`
- 텍스트 영역은 여러 줄의 둥근 사각형, 이미지는 정사각/원형 블록
- 데이터 도착 시 fade-in 트랜지션으로 자연스럽게 대체

3.3 낙관적 업데이트 Optimistic Update

설명	서버 응답을 기다리지 않고 UI를 먼저 업데이트한 뒤, 실패 시 롤백하는 패턴.
UX 원칙	도허티 임계값 — 400ms 이내 응답이 최적. 신뢰 설계 — 대부분의 요청이 성공한다면 성공을 기본 가정.
플랫폼	Both
구현 복잡도	High
실제 사례	Instagram 좋아요, Twitter/X 트윗 게시, Slack 메시지 전송

구현 팁

- 실패 시 롤백 로직 필수 — 이전 상태 스냅샷 보관
- TanStack Query의 `useMutation + onMutate`(낙관적 업데이트) + `onError`(롤백)
- 결제, 삭제 등 되돌리기 어려운 작업에는 적용 금지

3.4 프로그레스 인디케이터 Progress Indicator

설명	작업의 진행 정도를 시각적으로 보여주는 패턴. 결정형(퍼센트 표시)과 비결정형(무한 루프).
UX 원칙	목표 구매 효과 — 목표에 가까워질수록 행동이 빨라진다. 엔도워드 프로그레스 효과 — 이미 일부 진행된 것처럼 보여주면 완료 동기가 높아진다.
플랫폼	Both
구현 복잡도	Low
실제 사례	YouTube 영상 업로드, GitHub Actions, LinkedIn 프로필 완성도

구현 팁

- 전체 소요 시간 예측 가능하면 결정형, 예측 불가하면 비결정형
- 프로그레스 바는 끝에 가까워질수록 느려지는 것이 자연스러움
- 단계별 작업은 스텝 인디케이터로 표현
- <progress> HTML 요소 사용 — 기본 접근성 지원

3.5 토스트/스낵바 알림 Toast / Snackbar

설명	화면 상단 또는 하단에 일시적으로 나타났다가 사라지는 짧은 메시지.
UX 원칙	최소 방해 원칙 — 현재 작업을 중단시키지 않고 부가 정보 전달. 자동 해제 — 시간 기반 자동 소멸.
플랫폼	Both
구현 복잡도	Low
실제 사례	Gmail "메일이 전송되었습니다", Android Material Snackbar, VS Code 알림

구현 팁

- 표시 시간: 짧은 메시지 3초, 긴 메시지 5초, 액션 포함 시 8초
- 되돌릴 수 있는 작업에는 "실행취소(Undo)" 액션 버튼 포함
- role="status" 또는 aria-live="polite"로 접근성 확보
- 에러 토스트는 자동 소멸하지 않고 사용자가 직접 닫도록 설계

SECTION 04

제스처 기반 인터랙션 Gesture-based Interactions

터치 디바이스에서 제스처는 물리적 세계의 조작 경험을 디지털로 전이한 것이다. 잘 설계된 제스처는 직관적이고 효율적이지만, 발견 가능성이 낮으므로 항상 대안적 조작 수단을 병행해야 한다.

4.1 스와이프 액션 Swipe Actions

설명	리스트 아이템을 좌우로 스와이프하면 삭제, 보관, 읽음 처리 등의 빠른 액션이 나타나는 패턴.
UX 원칙	효율성 — 숙련 사용자에게 단축 경로 제공. 점진적 공개 — 기본 화면은 깔끔하게 유지.
플랫폼	Mobile
구현 복잡도	Medium
실제 사례	iOS Mail, Gmail, Todoist, 카카오톡

구현 팁

- 좌측 스와이프와 우측 스와이프에 서로 다른 액션 배치
- 삭제 액션은 빨간색, 보관은 파란색/회색 등 의미적 색상 코드 준수
- 실수 방지를 위해 삭제 시 "실행취소" 토스트 제공

4.2 핀치 투 줌 Pinch-to-Zoom

설명	두 손가락을 벌리거나 오므려 콘텐츠를 확대/축소하는 멀티터치 제스처.
UX 원칙	직접 조작 + 자연 매핑 — 손가락 사이 거리와 확대 비율의 직관적 대응.
플랫폼	Mobile
구현 복잡도	Medium
실제 사례	Google Maps, 사진 앱, Safari, Figma

구현 팁

- 최소/최대 줌 레벨 설정 (일반적으로 1x~5x)
- 더블탭으로 빠른 줌인/줌아웃 대안 제공

4.3 드래그 앤 드롭 Drag and Drop

설명	요소를 잡아 원하는 위치로 이동시키는 인터랙션. 순서 변경, 분류, 레이아웃 배치에 사용.
UX 원칙	직접 조작 + 공간적 기억 — 물리적 세계에서 물건을 옮기는 경험의 디지털 전이.
플랫폼	Both (데스크톱 우선)
구현 복잡도	High
실제 사례	Trello, Notion, Figma, 앱 아이콘 재배치

구현 팁

- 드래그 시작 시 원본에 반투명 복사본(ghost) 표시, 드롭 가능 영역 하이라이트
- 모바일에서는 long press(300~500ms)로 드래그 모드 진입
- 라이브러리: @dnd-kit/core, SortableJS, react-beautiful-dnd

4.4 롱 프레스 Long Press

설명	요소를 길게 누르면 컨텍스트 메뉴, 미리보기, 편집 모드 등이 활성화되는 패턴.
UX 원칙	점진적 공개 — 자주 사용하지 않는 기능을 숨겨 인터페이스를 단순화.
플랫폼	Mobile
구현 복잡도	Low
실제 사례	iOS Haptic Touch, WhatsApp, 카카오톡 메시지, Android 앱 아이콘 숏컷

구현 팁

- 롱 프레스 인식 시간: 300~500ms
- 롱 프레스로만 접근 가능한 필수 기능은 없어야 함

4.5 더블 탭 Double Tap

설명	동일 위치를 빠르게 두 번 탭하여 특정 액션을 실행하는 패턴.
UX 원칙	효율성 (Shortcut) — 숙련 사용자에게 빠른 접근 경로 제공.
플랫폼	Mobile
구현 복잡도	Low
실제 사례	Instagram 더블탭 좋아요, iOS 사진 앱 줌, YouTube 10초 이동

구현 팁

- 더블 탭 인식 간격: 200~300ms
- 접근성을 위해 동일 액션에 대한 버튼 대안 항상 제공

SECTION 05

스크롤 인터랙션

Scroll Interactions

스크롤은 웹과 모바일에서 가장 빈번한 인터랙션이다. 스크롤과 결합한 인터랙션은 콘텐츠 소비 경험을 풍부하게 만들지만, 성능 최적화가 핵심이다.

5.1 패럴랙스 스크롤링 Parallax Scrolling

설명	배경과 전경이 서로 다른 속도로 스크롤되어 깊이감을 만드는 시각 효과.
UX 원칙	깊이 인식 — 운동 시차는 깊이 인식 메커니즘을 활용. 몰입감 — 3차원적 경험 부여.
플랫폼	Web
구현 복잡도	Medium
실제 사례	Apple 제품 페이지, Nike, Spotify Year Wrapped

구현 팁

- CSS transform: `translate3d()`로 GPU 가속 활용
- 모바일에서는 패럴랙스 비활성화 또는 경량화
- `prefers-reduced-motion` 미디어 쿼리 존중
- Intersection Observer API로 뷰포트 진입 시에만 애니메이션 실행

5.2 무한 스크롤 Infinite Scroll

설명	페이지 하단에 도달하면 자동으로 다음 콘텐츠를 로딩하여 끊임 없는 탐색 경험 제공.
UX 원칙	흐름 상태 (Flow State) — 중단 없는 콘텐츠 스트림이 몰입 상태를 유지.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Instagram, Twitter/X, Pinterest, Facebook

구현 팁

- Intersection Observer로 "센티넬 요소"가 뷰포트 진입 시 다음 페이지 로딩
- "맨 위로" 버튼(FAB) 항상 제공
- 뒤로가기 후 복귀 시 이전 스크롤 위치 복원 필수
- 탐색적 콘텐츠(피드)에는 무한 스크롤, 목적 지향 콘텐츠(검색 결과)에는 페이지네이션

5.3 고정 헤더 Sticky Header

설명	스크롤 시 헤더가 화면 상단에 고정되어 항상 접근 가능한 패턴.
UX 원칙	접근성 유지 — 핵심 내비게이션을 항상 사용 가능하게. 피츠의 법칙 .
플랫폼	Both
구현 복잡도	Low
실제 사례	네이버, Amazon, Medium, GitHub

구현 팁

- CSS `position: sticky; top: 0;`로 간단 구현
- 아래로 스크롤 시 숨김, 위로 스크롤 시 표시 (show-on-scroll-up)
- 모바일에서는 스크롤 시 컴팩트 헤더로 축소

5.4 스크롤 트리거 애니메이션 Scroll-triggered Animation

설명	특정 요소가 뷰포트에 진입할 때 애니메이션이 실행되는 패턴.
UX 원칙	주의 유도 — 움직임은 시각적 주의를 자동으로 끌어당긴다. 점진적 공개 .
플랫폼	Web
구현 복잡도	Medium
실제 사례	Apple, Stripe, 토스, Airbnb

구현 팁

- Intersection Observer API (threshold: 0.1~0.3)
- 라이브러리: AOS, Framer Motion, GSAP ScrollTrigger
- 애니메이션은 한 번만 실행
- prefers-reduced-motion 대응 필수

5.5 스크롤 스냅 Scroll Snap

설명	스크롤이 끝날 때 가장 가까운 콘텐츠 경계에 자동으로 정렬되는 패턴.
UX 원칙	정렬 원칙 — 깔끔한 정렬로 시각적 완성도 향상. 예측 가능성 .
플랫폼	Both
구현 복잡도	Low
실제 사례	Instagram 릴스, TikTok, App Store, 당근마켓

구현 팁

- CSS scroll-snap-type: x mandatory 또는 y mandatory
- 자식 요소에 scroll-snap-align: start | center | end
- 페이지네이션 닷으로 현재 위치와 전체 개수 표시

SECTION 06

폼 인터랙션 Form Interactions

폼은 사용자와 시스템 간 데이터 교환의 핵심 인터페이스이다. 잘 설계된 폼은 완료율을 극적으로 향상시킨다. Luke Wroblewski의 연구에 따르면 폼 최적화만으로 전환율이 25~40% 개선될 수 있다.

6.1 인라인 유효성 검사 Inline Validation

설명	사용자가 입력 필드를 벗어나거나 입력하는 동안 실시간으로 유효성을 검사하여 피드백을 제공.
UX 원칙	즉각적 피드백 — 오류를 즉시 발견하면 수정 비용이 낮아진다. 완료 시간 22% 단축, 에러 발생 22% 감소.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Gmail 가입, GitHub 저장소 이름 검증, 토스 계좌번호 입력

구현 팁

- 입력 중(onChange)이 아닌 포커스 이탈 시(onBlur)에 검증
- 비밀번호 강도 같은 "긍정적 피드백"은 실시간 표시 가능
- 에러: 빨간색 + × 아이콘 / 성공: 녹색 체크마크(✓)
- aria-describedby로 에러 메시지를 필드에 연결

6.2 플로팅 라벨 Floating Labels

설명	입력 필드의 placeholder가 입력을 시작하면 필드 위로 올라가 라벨로 변환되는 패턴.
UX 원칙	공간 효율성 — 라벨과 placeholder를 합쳐 수직 공간 절약. 맥락 유지 .
플랫폼	Both
구현 복잡도	Low
실제 사례	Material Design, Airbnb, 토스, Stripe

구현 팁

- transform: translateY(-20px) scale(0.75) + transition: 200ms
- <label> 요소에 for 속성 필수
- 라벨 크기가 줄어든 때 최소 12px 이상 유지

6.3 자동완성 Auto-complete

설명	사용자 입력에 따라 가능한 선택지를 실시간으로 제시하여 빠른 입력을 돕는 패턴.
UX 원칙	획의 법칙 — 관련 없는 선택지를 필터링하여 결정 시간 단축. 인식 vs 기억 .
플랫폼	Both
구현 복잡도	Medium
실제 사례	Google 검색, GitHub 이슈/멘션, Slack, 카카오 주소 API

구현 팁

- 최소 2~3글자 입력 후 제안 시작
- 디바운스 200~300ms 적용
- 키보드 탐색(↑↓) + Enter 선택 지원
- role="combobox", aria-autocomplete ARIA 패턴 적용

6.4 스마트 기본값 Smart Defaults

설명	사용자의 맥락(위치, 시간, 이전 입력)을 기반으로 가장 가능성 높은 값을 미리 채워주는 패턴.
UX 원칙	최소 노력 원칙 — 최소한의 에너지 사용. 기본값 효과 — 옵트아웃 대비 옵트인 참여율이 50~90% 낮음.
플랫폼	Both
구현 복잡도	Medium
실제 사례	항공권 예매 사이트, 배달앱 기본 배송지, 캘린더 기본 일정

구현 팁

- GPS 기반 국가/도시 자동 선택
- 이전 입력 패턴 학습: 최근 사용 주소, 결제 수단 우선 표시
- 기본값은 언제든지 변경 가능해야 함

6.5 단계별 마법사 Step-by-Step Wizard

설명	복잡한 폼을 여러 단계로 나누어 한 번에 하나씩 처리하는 패턴.
UX 원칙	칭킹 — 작은 덩어리로 나누면 인지 부하 감소. 목표 구매 효과 + 완료 편향 .
플랫폼	Both
구현 복잡도	Medium
실제 사례	TurboTax, Typeform, 토스 계좌 개설, Airbnb 숙소 등록

구현 팁

- 단계 수를 미리 표시하여 전체 분량 예측 가능하게
- 각 단계는 3~5개 필드 이하로 제한
- 마지막 단계에서 전체 입력 요약(review) 화면 제공
- 부분 저장(draft) 기능으로 중간 이탈 후 복귀 가능하게

6.6 입력 마스크 Input Mask

설명	전화번호, 카드번호 등 특정 형식의 입력을 자동으로 포맷팅하는 패턴.
UX 원칙	오류 예방 — 오류 메시지보다 애초에 오류가 발생하지 않도록 설계하는 것이 우월.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Stripe 카드 결제, 토스 계좌번호 입력, 네이버 휴대폰 인증

구현 팁

- 숫자만 필요한 필드에는 `inputmode="numeric"`
- 커서 위치가 올바르게 유지되도록 주의
- 라이브러리: `cleave.js`, `react-input-mask`, `imask`
- 서버 전송 시에는 포맷 문자를 제거한 순수 값 전송

SECTION 07

전환 및 애니메이션 Transition & Animation

애니메이션은 사용자의 인지 모델을 돕는 도구이다. 어디서 왔고 어디로 가는지, 무엇이 변했는지를 설명하는 시각적 내려티브를 제공한다. 12가지 디즈니 애니메이션 원칙(1981)은 디지털 UI 애니메이션에도 유효하다.

7.1 페이지 전환 Page Transition

설명	페이지 간 이동 시 슬라이드, 페이드, 줌 등의 애니메이션으로 공간적 연속성 제공.
UX 원칙	공간적 일관성 — 페이지 간 공간적 관계를 애니메이션으로 전달. 연속성 원칙 .
플랫폼	Both
구현 복잡도	Medium
실제 사례	iOS UINavigationController, Material Design, Next.js page transitions

구현 팁

- 계층 이동(drill-down): 좌→우 슬라이드 진입, 우→좌 복귀
- 전환 시간: 200~300ms (모바일), 150~250ms (데스크톱)
- Web: View Transitions API 활용

7.2 공유 요소 전환 Shared Element Transition

설명	두 화면 간 공통 요소가 위치와 크기를 애니메이션하며 자연스럽게 연결되는 패턴.
UX 원칙	객체 항상성 — "새 페이지가 열린 것"이 아닌 "기존 요소가 확장된 것"으로 인식.
플랫폼	Both
구현 복잡도	High
실제 사례	Google Photos, Material Design 카드 확장, iOS App Store, Airbnb

구현 팁

- FLIP 기법 (First, Last, Invert, Play)
- React: Framer Motion의 `layoutId`, Flutter: Hero widget
- Web: View Transitions API의 `view-transition-name`

7.3 모핑 애니메이션 Morphing Animation

설명	한 형태가 다른 형태로 부드럽게 변화하는 애니메이션. 아이콘 변환, 버튼 형태 변경 등.
UX 원칙	연속성과 변환 — 상태 변화를 시각적으로 연결. 디즈니 원칙 "Squash and Stretch"의 디지털 적용.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Material Design 햄버거→화살표, iOS 잠금 해제, 음악 앱 재생/일시정지

구현 팁

- SVG path 모핑: 두 SVG path의 포인트 수를 동일하게 맞추어 `d` 속성 보간
- Lottie 파일로 복잡한 모핑 애니메이션 제작
- 라이브러리: `flubber`, `anime.js`, Lottie

7.4 스프링 물리학 Spring Physics

설명	기계적 이징 대신 스프링(질량, 강성, 감쇠) 기반 물리 시뮬레이션으로 자연스러운 움직임 구현.
UX 원칙	자연주의 — 실제 물리 법칙을 따르는 움직임. 디즈니 원칙: Follow Through .
플랫폼	Both
구현 복잡도	Medium
실제 사례	iOS UIKit Dynamics, Facebook/Meta, Framer Motion, Apple 위젯

구현 팁

- 핵심 파라미터: stiffness, damping, mass
- React: framer-motion의 type: "spring"
- React Native: react-native-reanimated의 withSpring

7.5 시차 애니메이션 Staggered Animation

설명	여러 요소가 일정 간격(50~100ms)을 두고 순차적으로 등장하는 패턴.
UX 원칙	시각적 계층 — 자연스러운 읽기 순서 안내. 디즈니 원칙: Staging .
플랫폼	Both
구현 복잡도	Low
실제 사례	Material Design 리스트, iOS 설정, Notion 사이드바, Slack

구현 팁

- animation-delay: `calc(index * 50ms)`
- 처음 5~10개 요소에만 시차 적용
- 기본: opacity: 0→1 + translateY: 20px→0

SECTION 08

온보딩 Onboarding

온보딩은 사용자가 제품의 가치를 최대한 빠르게 발견하도록 돕는 과정이다. 좋은 온보딩은 "가르치는 것"이 아니라 "경험하게 하는 것"이다. Samuel Hulick의 "더 나은 나를 위한 여정(Superhero Journey)" 프레임워크가 핵심 철학이다.

8.1 점진적 공개 Progressive Disclosure

설명	인터페이스의 기능을 한꺼번에 보여주지 않고 사용자의 진행도에 따라 단계적으로 노출.
UX 원칙	희의 법칙 — 선택지가 적을수록 결정이 빠르다. 인지 부하 감소.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Notion, Figma, Slack

구현 팁

- 첫 사용 시 3~5개 이하의 핵심 액션에 집중
- 사용자 행동 기반 트리거: 특정 기능을 N회 사용 후 관련 고급 기능 소개

8.2 툴팁 Tooltips

설명	UI 요소에 마우스를 올리거나 터치하면 해당 요소의 기능을 설명하는 작은 팝업.
UX 원칙	인식 vs 기억 — 필요할 때 도움을 받을 수 있다. 맥락적 도움.
플랫폼	Both
구현 복잡도	Low
실제 사례	GitHub, Figma, Jira, Notion

구현 팁

- 표시까지 200~500ms 딜레이
- 최대 2줄 이내의 짧은 텍스트
- `role="tooltip" + aria-describedby`

8.3 코치 마크 Coach Marks

설명	앱 처음 실행 시 주요 UI 요소를 하이라이트하고 순차적으로 설명하는 오버레이 안내.
UX 원칙	안내된 탐색 — 자유 탐색보다 구조화된 안내가 초기 학습에 효과적.
플랫폼	Both (모바일 앱에서 더 일반적)
구현 복잡도	Medium
실제 사례	Slack, Duolingo, 카카오톡 새 기능 소개, Instagram 릴스

구현 팁

- 최대 3~5단계로 제한
- "건너뛰기" 버튼 항상 제공
- 인터랙티브 방식이 수동적 읽기보다 학습 효과 높음

8.4

빈 상태 Empty States

설명	콘텐츠가 없는 초기 상태에서 안내 메시지, 일러스트레이션, CTA 버튼을 제공.
UX 원칙	행동 유도 — "다음 행동"을 명확히 안내하는 기회. 첫인상 효과 .
플랫폼	Both
구현 복잡도	Low
실제 사례	Dropbox, Trello, Mailchimp, 토스 증권

구현 팁

- 3가지 요소: 일러스트 + 설명 메시지 + CTA 버튼
- 빈 상태 유형별 대응: 첫 사용, 검색 결과 없음, 에러, 완료

8.5

기능 발견 Feature Discovery

설명	새로운 기능이나 업데이트된 기능을 사용자에게 알리는 패턴. 배지, 펄스 애니메이션, "What's New" 모달 등.
UX 원칙	자이가르닉 효과 — 미확인 배지가 탐색을 촉진.
플랫폼	Both
구현 복잡도	Low
실제 사례	iOS 앱 배지, Slack "새로운 기능" 배너, Figma "What's new"

구현 팁

- 강도 계층: 배지 점(낮음) < 인라인 배너(중간) < 모달(높음)
- 과도한 알림은 "알림 피로"를 유발하므로 신중하게 사용
- 변경 로그(Changelog) 페이지를 별도로 운영

SECTION 09

접근성 인터랙션 Accessibility Interactions

접근성은 선택이 아닌 필수이다. WCAG 2.1 AA 기준은 법적 요구사항인 경우가 많으며(한국 장애인차별금지법, 미국 ADA 등), 접근성 향상은 모든 사용자의 경험을 개선한다 (커브 컷 효과, Curb Cut Effect).

9.1 스크린 리더 패턴 Screen Reader Patterns

설명	시각 장애 사용자가 스크린 리더로 인터페이스를 탐색할 수 있도록 시맨틱 마크업과 ARIA 속성 제공.
UX 원칙	동등한 접근 — 모든 사용자가 동일한 정보와 기능에 접근 (WCAG 원칙 1: Perceivable).
플랫폼	Both
구현 복잡도	Medium
실제 사례	gov.uk, Apple, 정부24

구현 팁

- 시맨틱 HTML 우선: <button>, <nav>, <main>
- 이미지에 의미 있는 alt 텍스트, 장식용은 alt=""
- 동적 콘텐츠: aria-live="polite" 또는 "assertive"
- 아이콘 버튼에 aria-label 필수

9.2 포커스 관리 Focus Management

설명	키보드 내비게이션 시 포커스가 논리적 순서로 이동하고, 모달 시 포커스가 적절히 가둠(trap)되는 패턴.
UX 원칙	키보드 접근성 (WCAG 2.1.1). 포커스 트래핑.
플랫폼	Both (특히 Web)
구현 복잡도	Medium
실제 사례	Gmail, GitHub, VS Code, Jira

구현 팁

- 모달 열림: 포커스를 모달 첫 번째 인터랙티브 요소로 이동
- 모달 닫힘: 포커스를 트리거 요소로 복원
- `:focus-visible` (키보드) vs `:focus` (모든 포커스) 구분

9.3 모션 감소 Reduced Motion

설명	전정 기능 장애, 편두통 등을 가진 사용자를 위해 불필요한 애니메이션을 줄이거나 제거.
UX 원칙	포용적 디자인 — 다양한 조건을 가진 사용자를 포용.
플랫폼	Both
구현 복잡도	Low
실제 사례	iOS 설정, macOS/Windows 접근성 설정, Apple.com

구현 팁

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

- 모션을 완전히 제거하지 말고 크로스페이드(fade)로 대체
- 자동 재생되는 비디오/GIF에 정지 버튼 제공

9.4 색상 대비 Color Contrast

설명	텍스트와 배경 간 충분한 명도 대비를 확보하여 저시력·색각 이상 사용자도 콘텐츠를 읽을 수 있도록.
UX 원칙	인식 가능성 — WCAG 1.4.3: 일반 텍스트 4.5:1, 큰 텍스트 3:1 최소 대비비.
플랫폼	Both
구현 복잡도	Low
실제 사례	Material Design, Apple HIG, 정부24

구현 팁

- WCAG AA: 일반 텍스트 4.5:1, 큰 텍스트(18px+) 3:1
- WCAG AAA: 일반 텍스트 7:1, 큰 텍스트 4.5:1
- 색상만으로 정보를 전달하지 말 것 — 아이콘, 패턴, 텍스트 라벨 병행
- 고대비 모드: @media (forced-colors: active)

9.5 터치 타겟 크기 Touch Target Sizing

설명	인터랙티브 요소의 터치/클릭 가능 영역을 충분히 크게 설정.
UX 원칙	피츠의 법칙 — 타겟이 클수록, 가까울수록 빠르고 정확하게 접근.
플랫폼	Both (특히 Mobile)
구현 복잡도	Low
실제 사례	iOS HIG (44x44pt), Material Design (48x48dp), WCAG 2.5.8

구현 팁

- 최소: 44x44pt (iOS) / 48x48dp (Android) / 24x24px (WCAG AA)
- 시각적 크기가 작아도 padding으로 터치 영역 확장
- 인접한 터치 타겟 간 최소 8dp 간격
- min-height: 44px; min-width: 44px; 기본 스타일 포함

SECTION 10

딜라이트 패턴 Delight Patterns

딜라이트는 기능적 요구사항을 넘어 감성적 만족을 제공하는 디자인 요소이다. Don Norman의 "감성 디자인" 3단계 중 "반사적(Reflective)" 수준에 해당하며, 브랜드 차별화와 사용자 충성도에 기여한다. 단, 기본적인 사용성이 확보된 후에 적용해야 한다.

10.1 이스터 에그 Easter Eggs

설명	사용자가 우연히 또는 의도적으로 발견하는 숨겨진 기능이나 메시지.
UX 원칙	가변 보상 — 예측할 수 없는 보상이 탐색 동기를 높인다. 발견의 기쁨.
플랫폼	Both
구현 복잡도	Low~Medium
실제 사례	Google "do a barrel roll", Chrome 공룡 게임, Slack 로딩 메시지, VS Code Konami Code

구현 팁

- 핵심 기능에 영향을 주지 않는 곳에 배치
- 발견하지 못해도 제품 사용에 지장 없도록 설계

10.2 축하 애니메이션 Celebration Animation

설명	목표 달성, 과제 완료 시 컨페티, 불꽃놀이, 배지 획득 등의 시각적 축하 효과.
UX 원칙	긍정적 강화 — 바람직한 행동 후 보상 제공. 피크-엔드 법칙 — 경험의 정점과 끝이 전체 인상을 결정.
플랫폼	Both
구현 복잡도	Medium
실제 사례	Duolingo, GitHub 첫 PR 머지 컨페티, Notion 체크리스트, 토스

구현 팁

- canvas-confetti 라이브러리로 컨페티 효과
- 효과는 1~3초 이내로 짧게, 정말 의미 있는 성취에만 적용
- prefers-reduced-motion 대응: 정적 배지나 메시지로 대체

10.3 게이미피케이션 Gamification

설명	포인트, 배지, 레벨, 리더보드, 스트릭 등 게임 메커니즘을 비게임 환경에 적용.
UX 원칙	자기결정이론 — 자율성, 유능감, 관계성 3가지 내재적 동기 충족 시 지속적 참여.
플랫폼	Both
구현 복잡도	High
실제 사례	Duolingo, LinkedIn 프로필 완성도, GitHub Contributions, 토스 만보기

구현 팁

- 스트릭이 깨질 때 "프리즈" 기능 제공
- 외재적 보상만으로는 장기 참여 유지 어려움 — 내재적 동기와 연결 필수

10.4 개인화 경험 Personalized Experience

설명	사용자의 행동, 선호도, 위치, 시간 등을 기반으로 콘텐츠와 인터페이스를 맞춤화.
UX 원칙	칵테일 파티 효과 — 자신과 관련된 정보에 자동으로 주의 집중. 자기 참조 효과 .
플랫폼	Both
구현 복잡도	High
실제 사례	Spotify Discover Weekly, Netflix, YouTube, 토스, 쿠팡

구현 팁

- 개인화의 이유를 투명하게 설명
- 개인화 해제/조정 옵션 제공 (사용자 통제감 유지)
- 개인정보 보호와 투명성 최우선 — GDPR, 개인정보보호법 준수

10.5 사운드 디자인 Sound Design

설명	UI 인터랙션에 적절한 효과음을 결합하여 다중 감각적 경험을 제공.
UX 원칙	다중 감각 강화 — 시각+청각 결합 시 인지 정확도 향상. 소닉 브랜딩 .
플랫폼	Both (모바일에서 더 적극적)
구현 복잡도	Medium
실제 사례	카카오톡 알림음, 토스 송금 완료음, iOS 키보드 클릭음, Slack 알림음

구현 팁

- 소리는 기본 OFF — 사용자가 원할 때만 활성화
- 효과음은 100~500ms 이내의 짧고 경쾌한 소리
- Web Audio API 또는 Howler.js로 구현

SECTION 11

다크 패턴 — 피해야 할 것들 Dark Patterns to Avoid

다크 패턴(Deceptive Patterns)은 사용자를 속이거나 의도하지 않은 행동을 하도록 유도하는 사기적 디자인 기법이다. Harry Brignull이 2010년 처음 명명했으며, EU의 디지털 서비스법(DSA), 미국 FTC, 한국 공정거래위원회에서 점차 규제를 강화하고 있다.

11.1 확인 수치심 Confirmshaming

설명	거부 선택지를 죄책감을 느끼게 하는 문구로 작성하여 수락을 유도.
왜 나쁜가	사용자의 자율적 의사결정을 감정적으로 조작한다. 단기 클릭률 향상, 장기적 반감과 불신 축적.
실제 사례	"아니요, 할인 혜택을 포기하겠습니다", "저는 건강에 관심이 없습니다"

올바른 대안

- ✓ 거절 선택지는 중립적 문구: "괜찮아요, 다음에 할게요"
- ✓ 수락과 거절 버튼의 시각적 무게를 균등하게

11.2 강제 연속 Forced Continuity

설명	무료 체험 종료 후 명확한 통지 없이 자동으로 유료 구독으로 전환.
왜 나쁜가	사용자의 명시적 동의 없이 결제 발생. 한국 전자상거래법, EU 소비자 보호법에서 규제.
실제 사례	무료 체험 시 카드 등록 → 알림 없이 유료 전환, 해지 시 전화 필수 요구

올바른 대안

- ✓ 체험 종료 3일, 1일 전 이메일/앱 알림
- ✓ 구독 해지는 가입만큼 쉽게 — 설정에서 2클릭 이내
- ✓ 해지 후에도 남은 기간까지 서비스 이용 가능

11.3 트릭 질문 Trick Questions

설명	이중 부정, 혼란스러운 문구, 체크박스 의미 뒤집기 등으로 의도와 반대 선택을 유도.
왜 나쁜가	사용자의 인지적 취약점을 악용. 정보의 명확성 원칙에 위배.
실제 사례	"마케팅 이메일을 받지 않으려면 체크를 해제하지 마세요" (이중 부정)

올바른 대안

- ✓ 단순하고 긍정적인 문장: "마케팅 이메일을 받겠습니다" (체크=동의)
- ✓ 모든 체크박스의 의미 방향을 일관되게 유지
- ✓ 이중 부정 금지

11.4 숨겨진 비용 Hidden Costs

설명	결제 마지막 단계에서 이전에 공개되지 않았던 수수료, 배송비 등이 갑자기 추가.
왜 나쁜가	가격 투명성에 대한 신뢰를 파괴. 장바구니 포기의 1위 원인(48%, Baymard Institute).
실제 사례	항공권 가격에 세금·수수료 미포함, 배송비가 결제 단계에서만 표시

올바른 대안

- ✓ 상품 목록 단계부터 총 비용(배송비 포함) 표시
- ✓ 장바구니에서 비용 내역 항목별 투명하게 표시
- ✓ 무료 배송 기준 금액을 명확히 안내

11.5 방해 광고 / 강제 행동 Roach Motel

설명	가입·구독·동의를 매우 쉽게 만들면서 해지·탈퇴·취소는 의도적으로 어렵게 만드는 비대칭 설계.
왜 나쁜가	사용자의 선택의 자유를 구조적으로 제한. 장기적으로 네거티브 리뷰, 규제 기관 신고, 소송.
실제 사례	이메일 구독은 1클릭이지만 해지는 로그인 필요, 계정 삭제를 고객센터 전화로만 가능하게 함

올바른 대안

- ✓ 대칭적 설계: 가입이 3단계면 해지도 3단계 이하
- ✓ 이메일 하단 "수신 거부"는 1클릭으로 즉시 처리
- ✓ 해지 사유를 물어볼 수는 있지만 선택 사항으로

11.6 시각적 간섭 Visual Interference

설명	시각적 디자인으로 특정 선택지를 강조하거나 다른 선택지를 눈에 띄지 않게 만드는 패턴.
왜 나쁜가	시각적 계층 구조를 조작하여 실질적 선택의 자유를 제한. 특히 개인정보 동의에서 법적 문제.
실제 사례	"동의" 버튼은 크고 파란색, "거부" 버튼은 작은 회색 텍스트 링크

올바른 대안

- ✓ 동의/거부 버튼의 시각적 무게(크기, 색상, 위치)를 균등하게
- ✓ 쿠키 배너: "모두 수락" / "모두 거부" / "설정 관리" 3가지를 동일한 시각적 무게로

11.7 강제 행동 Forced Action

설명	원하는 기능을 사용하기 위해 관련 없는 행동(앱 평가, 소셜 공유 등)을 강제.
왜 나쁜가	사용자의 자율성을 침해. 강제된 행동의 데이터 신뢰성 훼손. Apple/Google Play 모두 리뷰 강제 금지.
실제 사례	"앱을 평가하지 않으면 이 기능을 사용할 수 없습니다", "친구 5명 초대해야 잠금 해제"

올바른 대안

- ✓ 평가 요청은 적절한 시점에 정중하게, 거절 시 다시 묻지 않기
- ✓ 선택적 공유: 공유하면 보너스, 하지 않아도 기본 기능 완전 사용 가능
- ✓ 데이터 수집은 최소 필요 범위만, 수집 사유 투명하게 설명

APPENDIX

UX 원칙 요약 사전

원칙	설명	제안자/출처
피츠의 법칙	타겟까지 이동 시간은 거리에 비례하고 크기에 반비례한다	Paul Fitts (1954)
Hick의 법칙	선택지가 많을수록 결정 시간이 로그 함수적으로 증가한다	William Hick (1952)
야콥의 법칙	사용자는 다른 사이트에서의 경험을 기반으로 디자인을 기대한다	Jakob Nielsen
밀러의 법칙	인간의 작업 기억은 한 번에 7±2개 항목을 처리한다	George Miller (1956)
도허티 임계값	시스템 응답이 400ms 이내일 때 생산성이 급격히 향상된다	Walter Doherty (1982)
목표 구배 효과	목표에 가까워질수록 노력을 높인다	Clark Hull (1932)
피크-엔드 법칙	경험의 정점과 끝이 전체 인상을 결정한다	Daniel Kahneman
폰 레스토르프 효과	유사한 항목들 중 눈에 띄는 것이 더 잘 기억된다	Hedwig von Restorff (1933)
자이가르닉 효과	미완료 과제가 완료된 과제보다 더 잘 기억된다	Bluma Zeigarnik (1927)
심미적 사용성 효과	미적으로 아름다운 디자인이 더 사용하기 쉽다고 인식된다	Kurosu & Kashimura (1995)
테슬러의 법칙	모든 프로세스에는 줄일 수 없는 최소한의 복잡성이 존재한다	Larry Tesler
직렬 위치 효과	목록의 첫 번째와 마지막 항목이 가장 잘 기억된다	Hermann Ebbinghaus (1885)
인지 부하 이론	인지 자원은 한정적이므로 불필요한 부하를 최소화해야 한다	John Sweller (1988)

참고 문헌

Norman, D. (2004). *Emotional Design: Why We Love (or Hate) Everyday Things*

Saffer, D. (2013). *Microinteractions: Designing with Details*

Krug, S. (2014). *Don't Make Me Think, Revisited*

Eyal, N. (2014). *Hooked: How to Build Habit-Forming Products*

Wroblewski, L. (2008). *Web Form Design: Filling in the Blanks*

Nielsen, J. (1994). *10 Usability Heuristics for User Interface Design*

Brignull, H. (2010). *Dark Patterns: Deception vs. Honesty in UI Design*

Material Design Guidelines — material.io

Apple Human Interface Guidelines — developer.apple.com/design

Web Content Accessibility Guidelines (WCAG) 2.1 — [w3.org/WAI](https://www.w3.org/WAI)

Laws of UX — lawsofux.com